1. Cardinality

Definition

Cardinality describes how many rows in one table relate to rows in another table.

1.1 One-to-One (1:1)

Example (Sakila):
Each store is managed by exactly one staff member.

- store.manager_staff_id → staff.staff_id

1.2 One-to-Many (1:N)

Example (Sakila):
One customer can have many rentals.

- customer.customer_id → rental.customer_id

1.3 Many-to-Many (M:N)

Example (Sakila):
Many actors appear in many films, handled through film_actor.

- actor.actor_id ↔ film_actor.actor_id
- film.film_id ↔ film_actor.film_id

Industry Use

- Data modeling
- Index and FK decisions
- Efficient ETL and query planning

2. SQL Joins

All examples use Sakila tables such as customer, rental, payment, film, and inventory.

2.1 INNER JOIN

Returns rows where both tables have matches.

Example (Customers with rentals):

sql

SELECT c.first_name, r.rental_dateFROM customer cINNER JOIN rental r ON c.customer_id = r.customer_id;

2.2 LEFT JOIN

Returns all rows from left table, matched or not.

Example (Customers without rentals):

sql

SELECT c.customer_id, r.rental_idFROM customer cLEFT JOIN rental r ON c.customer_id = r.customer_id;

2.3 RIGHT JOIN

(Sakila uses MySQL → less common, but possible.)
Used to check data inconsistencies.

2.4 FULL OUTER JOIN

Shows all rows from both tables; primarily supported in PostgreSQL.

2.5 CROSS JOIN

Creates all combinations.

Example:
All stores × all active promotions.

2.6 SELF JOIN

A table joins with itself.

Example:
staff table to identify reporting structure.

3. Subqueries

3.1 Scalar Subquery

Returns one value.

Example:
Films priced above the average rental rate.

sql

SELECT title, rental_rateFROM filmWHERE rental_rate > (SELECT AVG(rental_rate) FROM film);

3.2 Row Subquery

Filters using multiple columns.

3.3 Table Subquery

Used as a derived table.

3.4 Correlated Subquery

Evaluated per row.

Example: Identify customers whose payment is above their own average:

sql

```
SELECT p.*FROM payment pWHERE p.amount > (  SELECT AVG(amount)  FROM payment
WHERE customer_id = p.customer_id);
```

Industry Use:

- Analytical reporting
- Dynamic filters
- Security filtering
- yes

Table A	
1	A1
2	B3
4	A4

Vale B	
2	A1
3	A3
4	44

INNER JOIN

A.ID	AValue	B.ID	B.ID	Balue	Value
1	A2	2	2	B2	B2
2	A3	3	3	B3	B4
1	A3	3	4	B4	NULL
NULL	A4	6	5	NULL	NULL

LEFT JOIN

A.ID	AValue	B.ID	B.ID	Balus	Value
1	A2	2	3	B2	B4
2	A3	3	3	B4	B4
NULL	A3	6	5	B4	NBS

CROSS JOIN

A.ID	AValue	Value	B.ID	Balus	Value
1	A1	305	164	500	345
A1	A1	505	204	500	369
A1	A2	505	205	300	203
A3	A3	505	266	500	458
A9	A2	505	208	300	355
A3	A5	505	248	500	465
A9	A3	506	205	500	460
321	A2	505	207	500	267
521	A2	205	206	500	268
47	A3	204	305	300	369
521	A3	506	366	500	256
521	A3	206	374	500	865

Table A	
2	A2
3	B3
4	B4

Valle B	
2	A2
3	A4
4	A4

LEFT JOIN

A.ID	AValue	B.ID	B.ID	Balus	Value
2	A2	2	2	B2	B2
3	A3	3	3	B3	B4
4	A3	3	4	B4	NBS
NULL	A4	6	5	NULL	NULL

FULL OUTER JOIN

A.ID	AValue	B.ID	B.ID	Balus	Value
1	A2	2	3	B2	B4
2	A3	3	4	B4	B4
NULL	A3	6	5	NULL	NBS

CROSS JOIN

A.ID	AVast	Value	B.ID	Value	Balus	Value
1	A1	A1	200	106	300	465
A1	A1	B2	208	206	300	445
A1	A2	B4	305	207	500	455
B2	A3	B4	306	200	300	465
B2	A3	B4	205	206	300	445
B1	A2	B4	204	205	300	415
501	A3	B4	206	209	500	260
501	A3	B4	205	209	503	252
322	A2	B0	205	303	300	200
321	A2	B4	200	305	208	205
321	A2	B0	206	240	300	215
504	A5	B4	208	306	308	255

Green: Matched Rows
Blue: Left-Only Rows
Orange-Only Rows
Gray: Excluded/All Combinations

RIGHT JOIN

A.ID	AValue	B.ID	B.ID	Balue	Value
2	A2	2	2	B2	B2
3	A3	3	3	B3	B4
4	A3	4	4	B4	NULL
NULL	A3	6	5	NULL	NBS

SELF JOIN

Table A1

Table A	
1	A1
2	A3
3	B3
5	B4

Table A2

Table B	
2	A2
3	B3
4	B4
5	A4

SELF JOIN

A1ID	AValue	A.ID	Balue	Value
1	2 A1	2	A1	4
1	2 A1	3	A2	4
1	3 A3	3	A3	4
1	3 A3	3	A3	4
2	3 A2	3	A3	4
2	4 A3	4	A3	4
3	3 A3	4	A4	4

4. Common Table Expressions (CTEs)

Definition

A temporary named result set defined for a single query.

4.1 Non-Recursive CTE

sql

WITH customer_payments AS (SELECT customer_id, SUM(amount) AS total_paid FROM payment GROUP BY customer_id)SELECT *FROM customer_paymentsWHERE total_paid > 50;

4.2 Recursive CTE

Used for hierarchical or graph-like data.

Use Cases

- Organizing complex queries

- Step-wise transformations
- Recursion (hierarchies, chains)

Drawbacks

- Re-evaluated each time (can be slow)
- Not materialized
- Cannot persist
- Performance unpredictable on large data

5. User-Defined Functions (UDFs)

Definition

Reusable logic packaged into a function.

Example

Return number of rentals by customer:

sql

```
CREATE FUNCTION total_rentals(cid INT)RETURNS INTRETURN (  SELECT COUNT(*) FROM rental
WHERE customer_id = cid);
```

Uses

- Reusable calculations
- Business rules enforcement

Drawbacks

- Scalar UDFs can slow queries
- Limited optimizer support
- Some databases restrict data-modifying UDFs

6. Temporary Tables

Definition

A session-local table stored in tempdb.

Example

sql

CREATE TEMPORARY TABLE temp_expensive_films ASSELECT * FROM film WHERE rental_rate > 3.99;

Uses

- ETL staging
- Breaking transformations into steps
- Handling large intermediate datasets

Drawbacks

- Uses disk I/O
- Must be cleaned up
- Not shared between sessions

7. Views

Definition

Saved SQL query treated like a table.

Example

sql

CREATE VIEW top_customers ASSELECT c.customer_id, SUM(p.amount) AS total_spentFROM customer cJOIN payment p ON c.customer_id = p.customer_idGROUP BY c.customer_id;

Uses

- Security abstraction
- Logical data modeling
- Reduce repeated SQL

Drawbacks

- Not materialized unless indexed
- Poor performance when nested
- Underlying table changes may break views

8. Comparison Summary

Feature	CTE	Temp Table	View	UDF
Lifetime	Query	Session	Permanent	Permanent
Materialized	No	Yes	No (unless indexed)	No
Performance	Medium	Fast on large sets	Depends	Slow (scalar)
Reusable	No	No	Yes	Yes
Typical Use	readability	ETL	abstraction	business rules

9. Join Visual Diagrams (Color-Coded)

These will be provided as a color-coded diagram image.
It will include:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- SELF JOIN

With color-coded:

- Green = matched rows
- Blue = left-only
- Orange = right-only
- Gray = excluded